SmartCrop Frost ��� backend

SmartCrop Frost — backend

Temperature alerting for orchards. LoRaWAN sensors → ChirpStack → ingest/alert service → SMS / email / webhook (SmartCrop Field later). Multi-tenant: each SmartCrop customer is a ChirpStack tenant and a `companies` row.

```
Gateway (LPS8N) ──wss://lns.smartcrop.com──▶ nginx ──▶ gateway-bridge ──MQTT──▶ ChirpStack
──MQTT──▶ ingest ──▶ Postgres

│

└──▶ Twilio / smtp2go / webhook
```

Runs on the SmartCrop VPS

Host port	Service	Exposure
443	nginx → `frost.smartcrop.com` → ChirpStack UI (127.0.0.1:8100)	public, TLS
443	nginx → `lns.smartcrop.com` → Basic Station (127.0.0.1:8103)	public, TLS — gateways
443	nginx → `grafana.frost.smartcrop.com` → Grafana (127.0.0.1:8102)	optional
1700/udp	Semtech UDP fallback	firewall to known gateway IPs only
127.0.0.1:1883	Mosquitto	localhost
127.0.0.1:5433	Postgres (`chirpstack` + `frost` DBs)	localhost (VPS has its own on 5432)
127.0.0.1:8101	ChirpStack REST API	via nginx `/rest/`

Install (first time)

```
cd /opt && unzip smartcrop-frost.zip && cd smartcrop-frost
cp .env.example .env && nano .env         # Twilio, smtp2go, WEBHOOK_SECRET
bash scripts/vps-install.sh               # stack, device-profile import, nginx sites
# DNS: frost.smartcrop.com + lns.smartcrop.com → VPS IP (lns = DNS-only / grey cloud)
certbot --nginx -d frost.smartcrop.com -d lns.smartcrop.com
```

ChirpStack setup (once per server)

1. `https://frost.smartcrop.com` → `admin` / `admin` → change the password.
2. **Network Server → Device Profiles → Add** — global profile, shared by all tenants: Name `Dragino D20-LB US915`, Region `US915`, Region config `us915_1`, MAC `1.0.3`, Rev `A`, expected uplink `300`. Codec tab → *JavaScript functions* → paste `codecs/dragino-d2x-lb-chirpstack-v4.js`.
3. **Network Server → API Keys → Add** `smartcrop-backend` (admin). Store the token — SmartCrop's backend will use it to provision tenants/devices later.

Onboarding a customer (per company)

ChirpStack side: 1. **Network Server → Tenants → Add** (or rename the default “ChirpStack” tenant): the company name. Can have gateways = yes. 2. Inside the tenant: **Applications → Add** `smartcrop-frost`. 3. **Gateways → Add** one per ranch (Gateway EUI from the LPS8N label). 4. **Applications → smartcrop-frost → Add device** per sensor: DevEUI, Join EUI, global D20-LB profile; then **OTAA keys → Application key**.

Frost DB side (`seed.sql` shows the pattern):

```
insert into companies (name, tenant_id) values ('Northern Fruit', '<tenant uuid>');
insert into ranches (company_id, name) ...;
insert into sensors (dev_eui, company_id, ranch_id, name, placement) ...;
```

With `AUTO_REGISTER=true`, an unknown DevEUI is created automatically under the company whose `tenant_id` matches the uplink (or `DEFAULT_COMPANY`), unassigned to a ranch, so no reading is lost before you file it.

Gateway configuration (LPS8N)

LoRa: US915, sub-band 2 (channels 8–15). Must match `region_us915_1.toml`.

LoRaWAN → Basic Station (recommended, works from any internet connection): - LNS URI: `wss://lns.smartcrop.com` - Authentication: *TLS server authentication* - Trust (CA) file: Let’s Encrypt root — download `isrgrootx1.pem` from letsencrypt.org/certificates and upload it

LoRaWAN → Semtech UDP (fallback): server = VPS IP, port 1700 up/down; then `ufw allow from <gateway public ip> to any port 1700 proto udp`.

Data model (v2)

Table	What
`companies`	customer; `tenant_id` links to ChirpStack
`ranches`	per company
`sensors`	DevEUI, company, ranch, placement — no thresholds
`sensor_status`	last seen / last reading per sensor
`readings`	every uplink
`subscribers`	person or system; sms / email / webhook; quiet hours
`subscriptions`	subscriber × (ranch **or** sensor) × low °C × clear °C × offline minutes
`alert_state`	per (subscription, sensor): low active, offline active
`alerts`	history, with delivered flag

Rule evaluation, per reading, for every matching subscription: - `temp ≤ low_temp_c` and not active → **low_temp** → active - `temp ≥ clear_temp_c` and active → **low_temp_clear** → inactive - no uplink for `offline_minutes` and `notify_offline` → **offline**; next uplink → **back_online** - inside the subscriber’s quiet hours → suppressed, state unchanged (fires after the window if still true)

Two people on the same sensor with different thresholds get different alerts at different times. Ranch-scoped subscriptions automatically cover sensors added later.

Notification channels (`.env`)

Channel	Config	Notes

sms	TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_FROM	
email	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_STARTTLS, SMTP_FROM	smtp2go: mail.smtp2go.com, 2525, STARTTLS yes
webhook	WEBHOOK_SECRET	signed JSON POST; the SmartCrop Field hook

Webhook payload:

```
{"type":"low_temp","dev_eui":"a8404146fe5fbe4f","sensor":"Blue Heron - Sensor1",
 "company":"Northern Fruit","ranch":"Blue Heron","placement":"low spot",
 "temp_c":-2.4,"temp_f":27.7,"subscription_id":3,"subscriber":"Juan",
 "message":"FROST ALERT ...","at":"2026-11-18T11:12:40+00:00"}
```

Header `X-Frost-Signature: sha256=HMAC_SHA256(WEBHOOK_SECRET, body)`. `type` ∈ `low_temp`, `low_temp_clear`, `offline`, `back_online`.

Testing without hardware

```
        docker compose exec mosquitto mosquitto_pub -t
'application/test/device/a8404146fe5fbe4f/event/up' \
          -m '{"deviceInfo":{"devEui":"a8404146fe5fbe4f","deviceName":"Sensor1"},"object":
{"TempC1":-3.5,"BatV":3.6},"rxInfo":
[{"gatewayId":"test","rssi":-90,"snr":8}],"fCnt":1,"time":"2026-11-01T10:00:00Z"}'
        docker compose logs --tail 5 ingest
```

Clean up afterwards: `delete from readings where gateway_id='test'; delete from alerts; delete from alert_state;`

Upgrading an existing v1 (smartcrop-temp) install

`migrations/001_v1_to_v2_subscriptions.sql` converts a v1 `frost` database in place and `scripts/upgrade-from-smartcrop-temp.sh` copies the Docker volumes across. Not needed for a fresh VPS install.

Hardening checklist

- `api.secret` in `configuration/chirpstack/chirpstack.toml` → long random string
- Mosquitto is localhost-only; add users before ever exposing it
- Change the Postgres and Grafana passwords in `docker-compose.yml`
- `ufw`: allow 22, 80, 443; UDP 1700 only from known gateway IPs